

Autonomous TurtleBot Patrol & Obstacle Avoidance

ROS Task

1. Task Objective

Build a modular ROS 2 software stack for a TurtleBot3 operating in a simulated Gazebo environment. The robot must autonomously patrol a predefined route, detect and avoid obstacles using LiDAR, respond to an emergency-stop service, and publish/log basic telemetry.

2. Required Environment

- Ubuntu 22.04 + ROS 2 Humble/ Ubuntu 24.04 + ROS 2 Jazzy/ or Ubuntu 26.04 + a compatible ROS 2 distribution. **(You can use any distribution of linux)**
- Gazebo / Gazebo Sim with TurtleBot3 simulation.
- TurtleBot3 Burger or Waffle Pi.
- Python (rclpy) or C++ (rclcpp).
- colcon build.
- RViz2 for optional visualization.

3. Task Requirements

3.1 ROS 2 Package

Create a ROS 2 package named: **turtlebot_patrol**

3.2 Required Nodes

lidar_obstacle_detector — Perception

- Subscribe to /scan (sensor_msgs/msg/LaserScan).
- Divide the 360° LiDAR scan into Front (-30° to +30°), Left (+30° to +90°), and Right (-90° to -30°).
- Calculate the minimum distance in each sector.
- Publish obstacle information on /obstacle_status.
- Flag an obstacle when the relevant distance is below 0.5 m.

patrol_controller — Decision Making & Control

- Subscribe to /obstacle_status, /odom, and /estop_state.
- Publish velocity commands to /cmd_vel.
- PATROL: follow a square route or configured waypoints.
- AVOID: when an obstacle is detected in front, stop, rotate toward the more open side, then resume patrol.
- HALTED: when emergency stop is active, immediately publish zero linear and angular velocity.

emergency_stop_server — Safety & Services

- Provide /trigger_estop using std_srvs/srv/SetBool or Trigger.

- ## telemetry_logger — Diagnostics & Logging

- ## 4. Controller Behavior

AVOID: Obstacle detected in front. Stop, choose the more open left/right sector, rotate, and continue patrol.

5. Topics & Services

Interface	Type	Used By	Purpose
/scan	sensor_msgs/msg/Laser Scan	lidar_obstacle_detector	LiDAR input
/obstacle_status	std_msgs/msg/Float32MultiArray or custom msg	patrol_controller	Obstacle distances/status
/odom	nav_msgs/msg/Odometry	patrol_controller, telemetry_logger	Robot position and motion
/cmd_vel	geometry_msgs/msg/Twist	patrol_controller → robot	Velocity command
/trigger_estop	std_srvs/srv/SetBool or Trigger	operator → emergency_stop_server	Activate/reset E-stop
/estop_state	std_msgs/msg/Bool	emergency_stop_server → patrol_controller	Current E-stop state
/robot telemetry	std_msgs/msg/String	telemetry_logger	Telemetry output

6. Required Package Structure

```
turtlebot_patrol/
├── CMakeLists.txt / setup.py
├── package.xml
├── README.md
├── config/
│   └── patrol_params.yaml
├── launch/
│   └── patrol_simulation.launch.py
├── turtlebot_patrol/
│   ├── __init__.py
│   ├── lidar_obstacle_detector.py
│   ├── patrol_controller.py
│   ├── emergency_stop_server.py
│   └── telemetry_logger.py
└── srv/ # Optional custom service files
```

7. Launch Requirement

Provide one launch file that starts the complete simulation and all required nodes.

```
ros2 launch turtlebot_patrol patrol_simulation.launch.py
```

- Launch TurtleBot3 in a Gazebo world containing obstacles.
- Start all four required nodes.
- Load parameters from patrol_params.yaml.
- The complete system should be runnable from the single launch command above.

8. Configuration

Use patrol_params.yaml for configurable values such as:

- Safety/obstacle distance threshold
- Linear and angular speeds
- Patrol waypoints or route dimensions
- Telemetry/battery parameters

9. Submission Requirements

- GitHub repository link containing the complete ROS 2 package, including all source code, configuration files, launch files, and required project files.
- Clean, understandable project structure.
- 1 minute screen recording demonstrating patrol, obstacle avoidance, and the emergency-stop service.
- README.md containing setup, build, launch, usage testing instructions and min 3 images.

10. Evaluation Criteria — 100 Points

Area	Points	Assessment
Motion & Obstacle Avoidance	30	Patrol behavior, LiDAR-based detection, collision avoidance, and rotation logic.
ROS 2 Architecture & Nodes	25	Correct publishers/subscribers/services and modular node separation.
Safety & Emergency Service	15	Immediate halt on E-stop and correct resume after reset.
Code Quality & Modularity	15	Readable Python/C++, OOP where appropriate, ROS naming conventions, edge-case handling.
Documentation & Launch Setup	10	Clear README and working single-command launch.
Bonus / Extra Credit	5	RViz obstacle markers and/or Docker setup.

11. Expected Demonstration

1. Start the complete system using the unified launch command.
2. Show the robot moving through its patrol route.

3. Place or encounter an obstacle and demonstrate automatic avoidance.
4. Call `/trigger_estop` with `true` and demonstrate immediate stopping.
5. Call `/trigger_estop` with `false` and demonstrate resumed operation.
6. Show telemetry output and the generated CSV log.

Reference Videos

You may refer to the following video and playlist for guidance and examples:

Video:

<https://youtu.be/HJAE5Pk8Nyw>

Playlist (Complete Series):

<https://www.youtube.com/playlist?list=PLunhqkrRNRhYAffV8JDtFOatQXuU-NnxT>